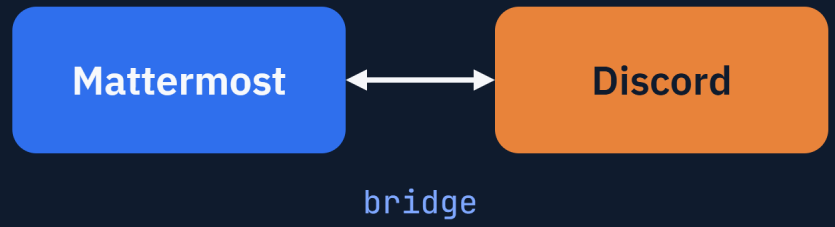
PROJET DOCKER · MASTÈRE 2 EXPERT FULL-STACK

IM Bridge

Relier Mattermost et Discord dans un stack Docker Compose sécurisé



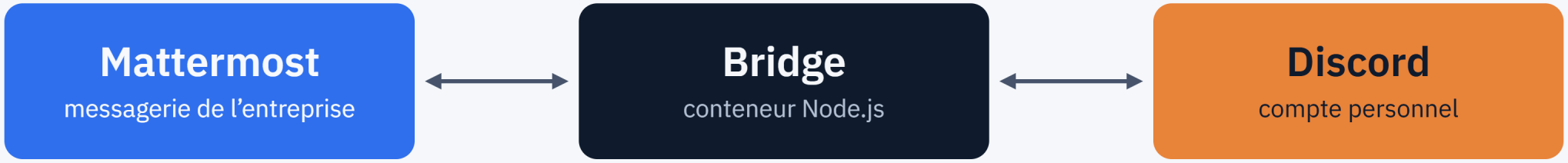
Safia Beaumale / Alexis Feron / Kirill Rosier

2026 – 2027

CONTEXTE

Une équipe, deux messageries

L’entreprise travaille sur Mattermost, mais une partie des utilisateurs reste sur Discord et ne veut pas changer d’outil.

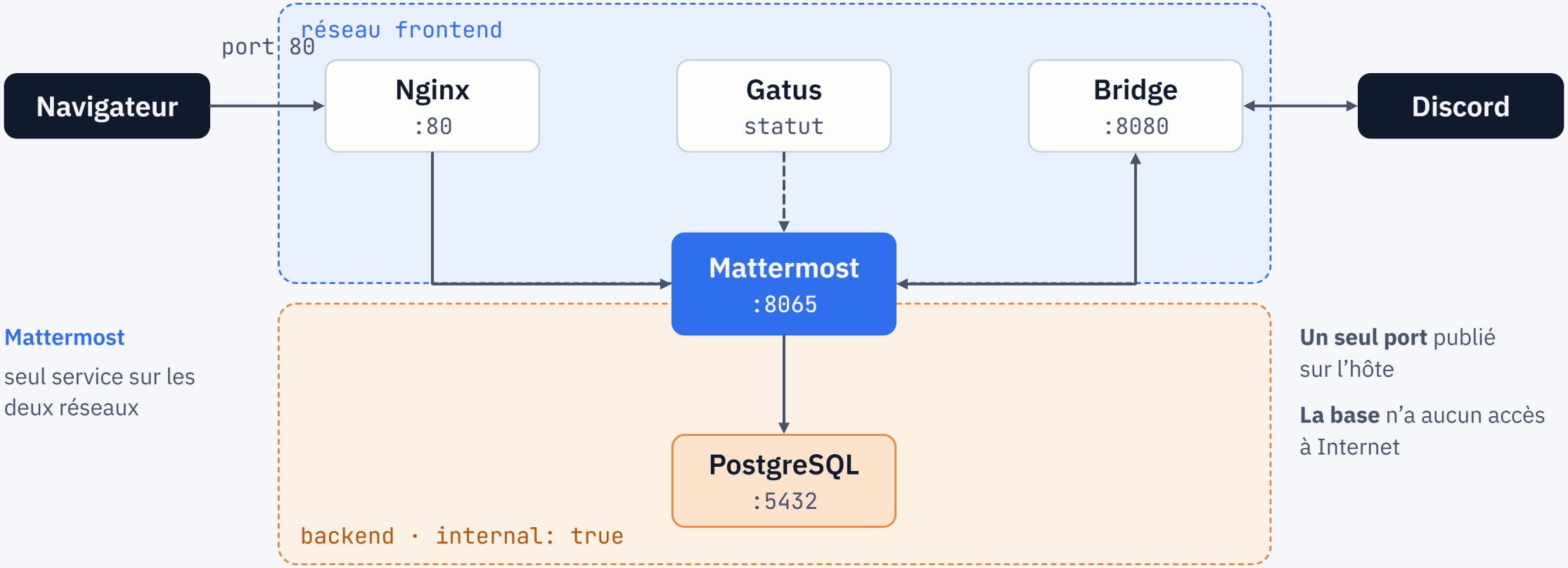


messages · réactions · fichiers · sondages, dans les deux sens

Objectif : chacun discute depuis et vers Mattermost avec son propre compte, sans quitter sa messagerie.

ARCHITECTURE

5 services, 2 réseaux



Mattermost

seul service sur les deux réseaux

Un seul port publié sur l'hôte

La base n'a aucun accès à Internet

NETWORKING

Qui peut parler à qui ?

Un seul port publié

Nginx sur le port 80. Mattermost, le bridge, Gatus et PostgreSQL restent sur les réseaux Docker.

Base enfermée

Réseau backend en internal: true : seul Mattermost la joint, aucune route vers Internet.

DNS interne

Les services s'appellent par leur nom. Nginx re-résout à chaque requête et route status.localhost vers Gatus.

```
$ docker compose exec db wget -T 3 http://example.com
wget: bad address 'example.com' → la base ne sort pas
```

LE BRIDGE

Un bridge écrit pour le projet

1 360

lignes de TypeScript : discord.js d'un côté, API REST et WebSocket de Mattermost de l'autre.

Matterbridge n'est plus maintenu et ne relaie ni réactions, ni stickers, ni sondages.

Messages
avec pseudo et avatar

Modifications
et suppressions

Réponses
dans le bon fil

Réactions
ajout et retrait

Fichiers
images, vidéos, jusqu'à 50 Mo

Vocaux
messages audio .ogg

GIF
Tenor, Giphy, fichiers

Stickers
Discord → Mattermost

Sondages
votes des deux côtés

LE BRIDGE

Parcours d’un message Discord

1

Alice poste sur Discord

Le bridge reçoit l’événement par la Gateway Discord.

2

Filtre anti-boucle

Ses propres messages sont ignorés, sinon ils rebondiraient à l’infini.

3

Fichiers ré-uploadés

Les liens du CDN Discord expirent : le fichier est copié dans Mattermost.

4

Post Mattermost

Créé par l’API REST, avec le pseudo et l’avatar d’Alice.

5

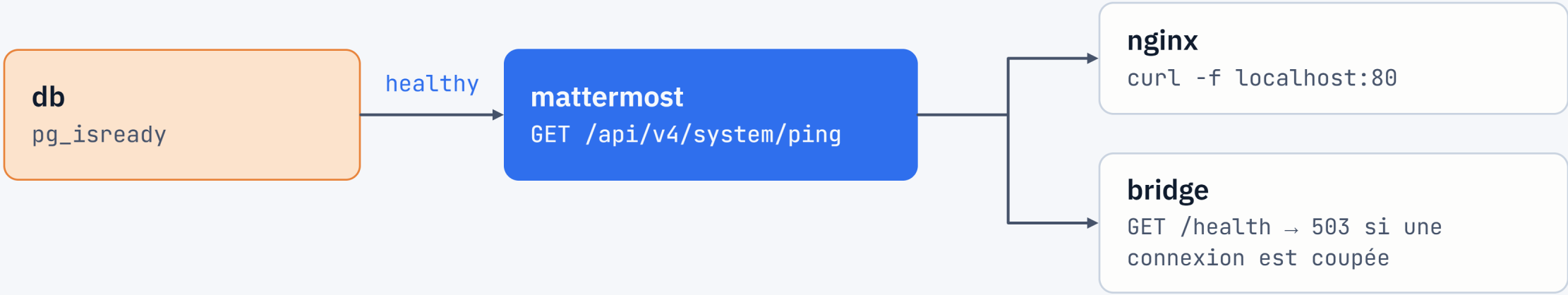
Store des IDs

ID Discord ⇄ ID Mattermost, gardé dans le volume bridge-data.

La clé, c’est le store : réponses, réactions, modifications et suppressions retrouvent le bon message de l’autre côté, même après un redémarrage.

HEALTHCHECKS

Un démarrage dans le bon ordre




```
depends_on:
  db:
    condition: service_healthy
```

Un conteneur démarré n’est pas un conteneur prêt.

Chaque service attend que ses dépendances soient healthy, et docker compose ps montre l’état réel.

IMAGES DOCKER

Une image sur mesure par service

STAGE builder · node:24-alpine

`npm ci → tsc → build-emoji → npm prune`



COPY --from=builder

IMAGE FINALE · node:24-alpine

dist/ + node_modules de production · USER node · HEALTHCHECK

Ni compilateur, ni sources, ni dépendances de dev : une image plus légère et moins de surface d'attaque.

Service	Ce qui change
db	Healthcheck pg_isready intégré
mattermost	Version en build arg, healthcheck, non-root
nginx	Configuration intégrée, version masquée, 50 Mo max
bridge	Build multi-stage, USER node, healthcheck
status	Image scratch : /data préparé dans un stage Alpine pour tourner en nobody

SÉCURITÉ

Défense en profondeur

Réseau

- 1 seul port publié
- backend internal
- bot Mattermost dédié

Conteneurs

- cap_drop: ALL
- no-new-privileges
- read_only + tmpfs
- non-root partout

Secrets

- .env hors de Git
- ${VAR:?} obligatoire
- aucun token dans le code

Ressources

- CPU, mémoire, pids
- rotation des logs
- requêtes ≤ 50 Mo

```
$ docker compose exec bridge sh -c 'id; touch /app/x'
uid=1000(node) … touch: /app/x: Read-only file system
```

COMPOSE AVANCÉ

Fragments et variables

Écrit une fois, hérité par les 5 services

```yaml
x-common: &common
  restart: unless-stopped
  logging: … max-size 10m, max-file 3

x-hardened: &hardened
  security_opt: [no-new-privileges:true]
  cap_drop: [ALL]

services:
  bridge:
    <<: [*common, *hardened]
```

Trois formes d'interpolation

Syntaxe	Effet
${VAR:?message}	Obligatoire : Compose refuse de démarrer
${VAR:-défaut}	Valeur par défaut : port, URL, TZ
${A}:${B}@db	Chaîne de connexion construite : mot de passe saisi une fois
build args	MATTERMOST_VERSION choisie dans .env

```
$ POSTGRES_PASSWORD= docker compose config
→ required variable POSTGRES_PASSWORD is missing a value
```

CGROUPS

Chaque service dans sa boîte

Service	CPU	Mémoire	pids	Au repos
mattermost	2	2 Go	1024	358 Mo
db	1	512 Mo	200	80 Mo
bridge	0,5	256 Mo	100	—
nginx	0,5	128 Mo	100	13 Mo
status	0,25	64 Mo	50	9 Mo

Fuite mémoire ?

Seul le service fautif est tué puis redémarré. Mattermost reste debout.

Fork bomb ?

La limite de processus la confine au conteneur.

deploy.resources.limits · « au repos » : docker stats sur une instance de test vide

ENVIRONNEMENT DE DEV

Un stack de dev en surcharge

```
$ docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build --watch
```

`docker-compose.yml`
la référence de production

+

`docker-compose.dev.yml`
uniquement les différences

↓

Stack de dev, fusionnée par Compose

Fichier modifié	Watch	Effet
nginx/nginx.conf	sync+restart	Nginx rechargé
status/config.yaml	sync+restart	Gatus rechargé
bridge/src/*.ts	rebuild	nouveau build multi-stage

ports sur 127.0.0.1

backend internal: false

logs DEBUG

Adminer avec --profile tools

DÉMONSTRATION · 3 MIN

On passe au live

01 docker compose ps : tout est healthy

02 status.localhost : tous les services au vert

03 Discord → Mattermost, avec pseudo et avatar

04 Réponse, réaction et image depuis Mattermost

05 !sondage, puis votes des deux côtés

06 Watch : on modifie nginx.conf, Nginx redémarre seul

Si on a 30 secondes de plus

```
docker compose config
docker network inspect docker_backend
docker stats --no-stream
docker image ls
```

BILAN

Ce que couvre le projet

- Mattermost, PostgreSQL et un bridge Discord
- Installation : un .env, puis docker compose up
- Variables obligatoires, par défaut, construites
- Réseaux isolés et conteneurs durcis
- Images modifiées, build multi-stage
- Stack de dev avec Compose Watch
- Fragments, cgroups, healthchecks

Pistes

- HTTPS sur Nginx
- Tokens en secrets Docker
- Visio : Mattermost Calls
- D’autres salons et messageries

Merci ! Des questions ?